Métodos de Análisis de Datos 1

Clase 7: APIs, Web Scraping y Tidy Data

Universidad Nacional del Sur

Contenidos de hoy

¿Qué es una API REST?

API: Application Programming Interface

Interfaz que permite comunicación entre aplicaciones

REST: REpresentational State Transfer

Estilo de arquitectura para servicios web

¿Qué es una API REST?

API: Application Programming Interface

Interfaz que permite comunicación entre aplicaciones

REST: REpresentational State Transfer

Estilo de arquitectura para servicios web

Ejemplos de APIs públicas

- OpenWeatherMap: datos meteorológicos
- Twitter/X: tweets, usuarios
- GitHub: repositorios, commits
- Google Maps: mapas, geocodificación
- The Movie Database: películas, series

Componentes de una API REST

1. **Endpoint (URL):** Dirección del recurso
   - `https://api.ejemplo.com/users`
2. **Métodos HTTP:**
   - GET: obtener datos
   - POST: crear nuevos datos
   - PUT/PATCH: actualizar datos
   - DELETE: eliminar datos
3. **Headers:** Metadatos (autenticación, tipo de contenido)
4. **Parámetros:** Query string o body
5. **Respuesta:** Generalmente JSON o XML

Anatomía de una request HTTP

Ejemplo completo

```
GET
https://api.openweathermap.org/data/2.5/weather?q=BahiaBlanca&appid=ABC123
```

- **Método:** GET
- **Base URL:** `https://api.openweathermap.org`
- **Endpoint:** `/data/2.5/weather`
- **Query params:**
  - `q=BahiaBlanca` (ciudad)
  - `appid=ABC123` (API key)

Anatomía de una request HTTP

Ejemplo completo

```
GET
https://api.openweathermap.org/data/2.5/weather?q=BahiaBlanca&appid=ABC123
```

- **Método:** GET
- **Base URL:** `https://api.openweathermap.org`
- **Endpoint:** `/data/2.5/weather`
- **Query params:**
  - `q=BahiaBlanca` (ciudad)
  - `appid=ABC123` (API key)

Respuesta típica (JSON)

```
{"temp": 25.3, "humidity": 60, "description": "clear sky"}
```

Códigos de respuesta HTTP

Éxito (2xx):

- **200 OK**: Exitoso
- **201 Created**: Recurso creado
- **204 No Content**: Sin contenido

Redirección (3xx):

- **301 Moved**: Movido permanentemente

Error del cliente (4xx):

- **400 Bad Request**: Solicitud inválida
- **401 Unauthorized**: No autenticado
- **404 Not Found**: No encontrado
- **429 Too Many**: Rate limit

Error del servidor (5xx):

- **500 Internal Error**: Error del servidor

GET Request en Python

```python
import requests
import pandas as pd

# Request simple
url = "https://api.ejemplo.com/posts"
response = requests.get(url)

# Verificar  xito
if response.status_code == 200:
    data = response.json()
    df = pd.DataFrame(data)
    print(df.head())
else:
    print(f"Error: {response.status_code}")

# Con par metros
params = {'userId': 1, 'limit': 10}
```

GET Request en R

```r
library(httr)
library(jsonlite)

# Request simple
url <- "https://api.ejemplo.com/posts"
response <- GET(url)

# Verificar  xito
if (status_code(response) == 200) {
  data <- content(response, as = "text")
  df <- fromJSON(data)
  head(df)
} else {
  print(paste("Error:", status_code(response)))
}

# Con par metros
```

POST Request (crear datos)

Python:

```python
import requests

url = "https://api.ejemplo.com/
    posts"

# Datos a enviar
nuevo_post = {
    'title': 'Nuevo post',
    'body': 'Contenido',
    'userId': 1
}

# POST request
response = requests.post(
    url,
    json=nuevo_post
```

R:

```r
library(httr)

url <- "https://api.ejemplo.com/
    posts"

# Datos a enviar
nuevo_post <- list(
  title = 'Nuevo post',
  body = 'Contenido',
  userId = 1
)

# POST request
response <- POST(
  url,
  body = nuevo_post,
```

Rate Limiting

Problema

Las APIs limitan el número de requests por tiempo

Límites típicos

- 100 requests por hora
- 1000 requests por día
- 10 requests por segundo

Rate Limiting

Problema

Las APIs limitan el número de requests por tiempo

Límites típicos

- 100 requests por hora
- 1000 requests por día
- 10 requests por segundo

Soluciones

1. Espaciar requests (sleep entre llamadas)
2. Cachear resultados
3. Pagar por plan premium
4. Usar múltiples API keys (con cuidado)

Implementar Rate Limiting

Python:

```python
import time
import requests

urls = [
    'url1', 'url2', 'url3'
]

resultados = []
for url in urls:
    response = requests.get(url)
    resultados.append(response.
        json())

    # Esperar 1 segundo
    time.sleep(1)
```

R:

```r
library(httr)

urls <- c('url1', 'url2', 'url3')

resultados <- list()
for (url in urls) {
  response <- GET(url)
  resultados[[length(resultados) +
      1]] <- content(response)

  # Esperar 1 segundo
  Sys.sleep(1)
}
```

¿Qué es web scraping?

Definición

Extraer datos de páginas web mediante programación

¿Cuándo usar?

SOLO cuando no hay API disponible

¿Qué es web scraping?

Definición

Extraer datos de páginas web mediante programación

¿Cuándo usar?

SOLO cuando no hay API disponible

Proceso

1. Descargar HTML de la página
2. Inspeccionar estructura (DevTools del navegador)
3. Identificar selectores CSS o XPath
4. Extraer información específica
5. Limpiar y estructurar datos

Herramientas de web scraping

Python:

- **BeautifulSoup**: Parsing HTML simple
- **Scrapy**: Framework completo
- **Selenium**: Páginas dinámicas (JavaScript)
- **requests-html**: Combina requests y BS4

R:

- **rvest**: Principal librería
- **xml2**: Parsing XML/HTML
- **RSelenium**: Páginas dinámicas

Recomendación

Comenzar con BeautifulSoup (Python) o rvest (R)

Selectores CSS básicos

¿Qué son?

Patrones para identificar elementos HTML

Selector	Descripción
`p`	Todos los párrafos
`.clase`	Elementos con clase="clase"
`#id`	Elemento con id="id"
`div p`	Párrafos dentro de divs
`div > p`	Párrafos hijos directos de divs
`a[href]`	Enlaces con atributo href
`.precio.destacado`	Elementos con ambas clases

Usar DevTools del navegador (F12) para inspeccionar elementos

Web Scraping en Python

```python
import requests
from bs4 import BeautifulSoup

# Descargar p gina
url = "https://ejemplo.com/productos"
response = requests.get(url)
soup = BeautifulSoup(response.content, 'html.parser')

# Extraer todos los enlaces
enlaces = soup.find_all('a')
for link in enlaces:
    print(link.get('href'))

# Extraer tabla
tabla = soup.find('table', {'class': 'datos'})
filas = tabla.find_all('tr')
```

Web Scraping en R

```r
library(rvest)

# Descargar p gina
url <- "https://ejemplo.com/productos"
pagina <- read_html(url)

# Extraer enlaces
enlaces <- pagina %>%
  html_nodes("a") %>%
  html_attr("href")

# Extraer tabla
tabla <- pagina %>%
  html_node("table.datos") %>%
  html_table()

df <- as_tibble(tabla)
```

Consideraciones legales y éticas

SIEMPRE verificar antes de scrapear

1. **Términos de servicio:** ¿Permiten scraping?
2. **robots.txt:** `https://sitio.com/robots.txt`
3. **Frecuencia:** No saturar el servidor
4. **User-Agent:** Identificarse apropiadamente
5. **Datos personales:** Respetar privacidad

Consideraciones legales y éticas

SIEMPRE verificar antes de scrapear

1. **Términos de servicio:** ¿Permiten scraping?
2. **robots.txt:** `https://sitio.com/robots.txt`
3. **Frecuencia:** No saturar el servidor
4. **User-Agent:** Identificarse apropiadamente
5. **Datos personales:** Respetar privacidad

Buenas prácticas

- Pausas entre requests (1-3 segundos)
- Cachear resultados
- Scrapear de noche (menos carga)
- Respetar directivas de robots.txt
- Contactar al sitio si tenés dudas

User-Agent y robots.txt

Configurar User-Agent:

```
# Python
headers = {
    'User-Agent': 'MiScraper/1.0 (contacto@ejemplo.com)'
}
response = requests.get(url, headers=headers)
```

Verificar robots.txt:

```
https://wikipedia.org/robots.txt

User-agent: *
Disallow: /wiki/Special:
Allow: /wiki/
```

Importante

Si robots.txt prohíbe, **NO scrapear**

Principios de Tidy Data

¿Qué son datos ordenados (tidy)?

Datos estructurados de forma consistente para facilitar análisis

Tres principios (Hadley Wickham)

1. **Cada variable forma una columna**
2. **Cada observación forma una fila**
3. **Cada tipo de unidad observacional forma una tabla**

Principios de Tidy Data

¿Qué son datos ordenados (tidy)?

Datos estructurados de forma consistente para facilitar análisis

Tres principios (Hadley Wickham)

1. **Cada variable forma una columna**
2. **Cada observación forma una fila**
3. **Cada tipo de unidad observacional forma una tabla**

¿Por qué importa?

- Facilita manipulación con dplyr/pandas
- Visualización más simple
- Menos errores
- Código más claro y mantenible

Ejemplo: Datos NO ordenados

Wide format (ancho):

Persona	2020	2021	2022
Ana	50000	52000	54000
Luis	48000	49000	51000

Problemas

- Los años están en columnas (deberían ser valores)
- Difícil agregar nuevos años
- Difícil visualizar series temporales

Ejemplo: Datos ordenados

Long format (largo):

Persona	Año	Ingreso
Ana	2020	50000
Ana	2021	52000
Ana	2022	54000
Luis	2020	48000
Luis	2021	49000
Luis	2022	51000

Ventajas

- Fácil filtrar por año
- Fácil agrupar y sumarizar
- Compatible con ggplot2/seaborn

Wide → Long (Pivot/Melt)

Objetivo

Convertir columnas en filas (wide → long)

Wide

Persona	2020	2021	2022
Ana	50k	52k	54k
Luis	48k	49k	51k

⟶

Long

Persona	Año	Ingreso
Ana	2020	50k
Ana	2021	52k

- Python: `pd.melt()`
- R: `pivot_longer()`

Wide → Long: Python

```python
import pandas as pd

# Datos wide
df_wide = pd.DataFrame({
    'persona': ['Ana', 'Luis'],
    '2020': [50000, 48000],
    '2021': [52000, 49000],
    '2022': [54000, 51000]
})

# Melt (wide      long)
df_long = pd.melt(
    df_wide,
    id_vars=['persona'],            # Columnas a mantener
    value_vars=['2020', '2021', '2022'],  # Columnas a convertir
    var_name='a  o',                 # Nombre de nueva columna
    value_name='ingreso'            # Nombre de valores
)
```

Wide → Long: R

```r
library(tidyverse)

# Datos wide
df_wide <- tibble(
  persona = c('Ana', 'Luis'),
  `2020` = c(50000, 48000),
  `2021` = c(52000, 49000),
  `2022` = c(54000, 51000)
)

# Pivot longer (wide      long)
df_long <- df_wide %>%
  pivot_longer(
    cols = c(`2020`, `2021`, `2022`),  # Columnas a convertir
    names_to = "a  o",                  # Nombre de columna
    values_to = "ingreso"               # Nombre de valores
  )
```

Long → Wide (Pivot)

Objetivo

Convertir filas en columnas (long → wide)

Long

Persona	Año	Ingreso
Ana	2020	50k
Ana	2021	52k

⟶

Wide

Persona	2020	2021	2022
Ana	50k	52k	54k
Luis	48k	49k	51k

- Python: `pivot()` o `pivot_table()`
- R: `pivot_wider()`

Long → Wide: Python

```
# Pivot (long      wide)
df_wide = df_long.pivot(
    index='persona',        # Columna para filas
    columns='a  o',          # Columna para columnas
    values='ingreso'        # Valores a rellenar
)

# Si hay duplicados, usar pivot_table con agregaci n
df_wide = df_long.pivot_table(
    index='persona',
    columns='a  o',
    values='ingreso',
    aggfunc='mean'  # Funci n de agregaci n
)

# Reset index para tener 'persona' como columna
df_wide = df_wide.reset_index()
```

Long → Wide: R

```
# Pivot wider (long     wide)
df_wide <- df_long %>%
  pivot_wider(
    names_from = a o ,       # Columna que se convierte en columnas
    values_from = ingreso   # Valores a rellenar
  )

# Con prefijo
df_wide <- df_long %>%
  pivot_wider(
    names_from = a o ,
    values_from = ingreso ,
    names_prefix = "a  o _"
  )

# Con m ltiples columnas de valores
df_wide <- df_long %>%
```

Resumen de la clase

- Las **APIs REST** proveen acceso estructurado a datos
- Componentes: endpoint, método HTTP, headers, parámetros, respuesta
- Códigos: 200 (OK), 401 (no autorizado), 404 (no encontrado)
- Implementar **rate limiting** para respetar límites
- **Web scraping**: último recurso cuando no hay API
- Herramientas: BeautifulSoup (Python), rvest (R)
- Respetar **robots.txt** y términos de servicio
- **Tidy data**: cada variable una columna, cada observación una fila
- **Wide → Long**: melt/pivot_longer
- **Long → Wide**: pivot/pivot_wider

Próxima unidad

Unidad 2: Exploración y Visualización de Datos

Tarea

1. **APIs:**
   - Elegir una API pública (ej: JSONPlaceholder, OpenWeather)
   - Hacer al menos 3 requests diferentes
   - Convertir respuestas a DataFrames
2. **Web scraping (opcional):**
   - Elegir un sitio simple (verificar robots.txt)
   - Extraer una tabla o lista
   - Estructurar en DataFrame
3. **Tidy data:**
   - Crear dataset wide
   - Convertir a long
   - Volver a wide
   - Visualizar ambos formatos
4. Documentar todo y subir a GitHub

¿Preguntas?